Feature Map Compression for Split Learning in Mobile Wireless Environment: Theory and Practice

Wenxuan Zhou, Zhihao Qu, *Member, IEEE*, Baoliu Ye, *Member, IEEE*, Shen-Huan Lyu, Miao Cai, *Member, IEEE*, Mingtao Ji, *Member, IEEE*, and Song Guo, *Fellow, IEEE*

***Abstract*—Split Learning (SL) enables collaborative model training by distributing computation between resource-constrained mobile edge devices and a central server. In this distributed framework, mobile edge devices compute the early layers of the model and transmit feature maps to the server. However, due to practical mobile wireless bandwidth limitations, these feature maps should be compressed before transmission. In this paper, we propose a lightweight Mask-Encoded Sparsification (MS) compressor that enhances Top-*k* sparsification with a narrow-bit mask, significantly reducing compression error while improving computational and communication efficiency. Our theoretical analyses reveal that feature map compression generally induces estimator bias in the gradients. Furthermore, we establish the relationships between compression error and convergence, as well as between the choice of cutlayer and output error. The first relationship demonstrates that reducing compression error facilitates convergence, confirming the advantage of MS, whereas the second provides theoretical guidance for determining the cutlayer, indicating that earlier layers are more sensitive to compression errors. Extensive experiments on vision and language models demonstrate that our method outperforms existing compression techniques in terms of compression error and model performance, achieving up to a 21.38× speedup in communication efficiency under wireless environments.**

***Index Terms*—Split learning, communication compression, sparsification, convergence theory.**

Received 28 October 2025; revised 2 April 2026; accepted 18 May 2026. Date of publication 25 May 2026; date of current version 2 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62441225, Grant 62572171, and Grant 62306104, in part by the Basic Research Program of Jiangsu under Grant BK20253011, in part by Hong Kong Scholars Program under Grant XJ2024010, in part by the Research Grants Council of the Hong Kong Special Administrative Region, China, under Grant CityU11212524, in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant JYB2025XDXM118, and in part by "111 Center" under Grant B26023. Recommended for acceptance by D. Li. *(Corresponding authors: Zhihao Qu; Baoliu Ye.)*

Wenxuan Zhou, Zhihao Qu, Shen-Huan Lyu, and Mingtao Ji are with the Key Laboratory of Water Big Data Technology of Ministry of Water Resources,Hohai University, Nanjing 210024, China (e-mail: quzhihao@hhu.edu.cn).

Baoliu Ye is with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210093, China (e-mail: yebl@nju.edu.cn).

Miao Cai is with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China.

Song Guo is with the Hong Kong University of Science and Technology, Hong Kong, SAR, China.

This article has supplementary downloadable material available at https://doi.org/10.1109/TMC.2026.3696694, provided by the authors.

Digital Object Identifier 10.1109/TMC.2026.3696694

I. Introduction

DRIVEN by emerging edge-intelligence applications in mobile wireless networks, such as on-device speech recognition and mobile healthcare monitoring, recent years have witnessed a growing demand for training deep neural networks (DNNs) [1], [2] on resource-constrained devices, including smartphones and embedded AI chips. Federated learning (FL) [3], [4], [5] has become a widely adopted distributed learning paradigm, allowing multiple devices to collaboratively train a shared global model without exchanging raw data. However, FL typically requires each device to execute full model training locally before uploading updates to the server. This design often underutilizes powerful server-side resources while imposing significant computational and energy burdens on resource-limited mobile edge devices. These issues are further amplified by the increasing depth and complexity of modern DNN architectures [1], thereby limiting the applicability of FL in low-resource mobile wireless environments.

To address this issue, Split Learning (SL) [6], [7], [8] has recently gained attention as a promising alternative that leverages the computational strengths of both end devices and central servers. SL divides a DNN into two parts: a client-side model deployed on the end device and a server-side model hosted on the central server. During forward propagation, intermediate feature maps (also referred to as *smashed data*) are transmitted from a designated cutlayer on the client side to the server, enabling the continuation of computation on server-side layers. In the backward pass, the partial gradients from the cutlayer are sent back to the clients to facilitate local updates. Through this collaborative scheme, SL allows resource-constrained mobile devices to participate in training large-scale DNNs.

A significant challenge in SL lies in the substantial communication cost [9] associated with transmitting feature maps, particularly during the forward pass. Unlike FL, where lazy aggregation [10] can be employed to reduce communication overhead linearly, the non-additive nature of feature maps prevents such techniques from being applied in SL. Consequently, the iterative exchange of smashed data becomes time-consuming and impractical, especially under limited mobile wireless bandwidth connections between the clients and server. In addition, compressing the feature map during the training phase enables the neural network to adapt to errors, reducing communication overhead during the inference phase. Thus, developing communication-efficient mechanisms is crucial for

1536-1233 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

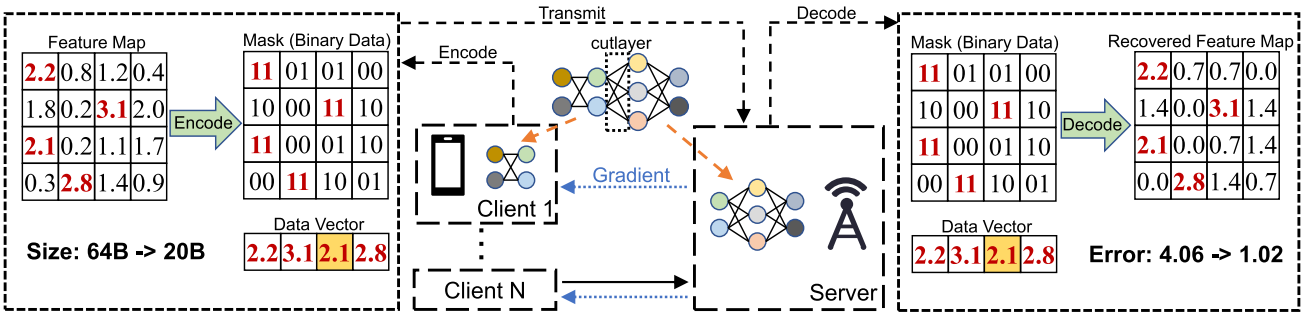


Fig. 1. Using the MS algorithm to compress smashed data in SL. Perform Top-4 sparsification on the smashed data, then compensate for filtered data with a 2-bit width mask. The top values (highlighted in red) form a data vector in their original order. Their corresponding mask is $0b11$. The filtered values are calculated with a quantization-like method, MS evenly divides 0 to 2.1 (the smallest top value, with a yellow background) into 3 ($3 = 2^2 - 1$) segments ($[0, 0.7)$, $[0.7, 1.4)$, and $[1.4, 2.1)$), the value in each segment will be rounded to 0, 0.7, and 1.4, and their masks are $0b00$, $0b01$, and $0b10$, respectively. Finally, the data vector is sent to the server along with the mask for decoding. All values with a mask of $0b11$ will be sequentially filled with the Top value, while the remaining masks will be filled with 0, 0.7, and 1.4, respectively. This strategy diminishes compression errors (measured by the ℓ_2 norm) from 4.06 to 1.02, achieving superior performance compared to vanilla sparsification.

enabling scalable SL in mobile wireless edge environments. While various communication reduction strategies such as quantization [11], [12] and sparsification [13], [14], [15] have been explored in FL, they are generally unsuitable for SL. These methods are primarily designed for compressing gradients or model parameters, rather than feature maps.

In this paper, we propose a new communication compression framework for SL, namely mask-encoded sparsification (MS). Based on Top-k sparsification, MS applies an encoded mask to compensate for the significant compression errors caused by sparsification. Specifically, the feature map is first sparsified by Top-k selection, after which an encoded mask is generated to indicate the position and value of every element in the feature map. For the Top-k elements, the mask is represented as a binary array of 1 s, while the remaining elements are encoded using a narrow-bit quantization method. As illustrated in Fig. 1, MS utilizes a narrow bit width mask (2-bit), effectively reducing the compression error caused by sparsification and being beneficial for convergence.

We subsequently conduct a series of theoretical analyses. Firstly, compressing feature maps, even with unbiased compression, generally introduces estimator bias in the computed gradients, negatively impacting convergence [16]. Secondly, by applying the mean value theorem, we establish the relationship between compression error and model convergence. Our analysis demonstrates that lower compression error leads to improved convergence, validating the superiority of MS. Lastly, we derive the relationship between the choice of cutlayer and the output error. This analysis reveals that shallower cutlayers are more sensitive to compression errors, resulting in larger output errors, thus providing guidance for the cutlayer selection in SL.

The main contributions of this work are summarized as follows:

- We propose the mask-encoded sparsification (MS) that significantly reduces the compression error of the feature map. Empirical results validate that MS achieves lower compression error compared to existing methods.
- We theoretically demonstrate that compressing feature maps in SL generally introduces biased gradients (Theorem 1). By applying the mean value theorem, we establish the relationship between compression error and convergence (Theorem 3), indicating that lower compression error leads to better convergence, verifying the effectiveness of MS.
- We establish the relationship between the cutlayer and output error (Theorem 4), showing that shallower cutlayers result in larger errors in both forward and backward propagation. This observation is also supported by experimental evidence.
- Our experiments across diverse models and datasets show that MS achieves efficient encoding and decoding while significantly reducing compression error.[1] Furthermore, it yields a significant improvement in communication efficiency under wireless environments (up to a 21.38× speedup). Compared to prior methods that severely degrade model performance, MS maintains near-baseline accuracy and achieves minimal perplexity degradation in both vision and language tasks.

II. Related Work

A. Split Learning

SL [6], [17], [18] divides a deep neural network into two contiguous segments, referred to as the client-side and server-side models [19], [20]. The former is executed locally on resource-constrained edge devices, while the latter is hosted on a central server.

In a typical SL workflow, during forward propagation, each client processes its local data through the client-side model to generate intermediate feature maps. These feature maps are transmitted to the server, which completes the forward pass using the server-side model to produce predictions. During backward propagation, the server computes the gradients with respect to the received feature maps and sends them to the client, which completes the backward pass for the client-side model. The server and the client then update their respective model

[1] Code is available at https://github.com/BinaryMus/MaskSparsification/tree/journal.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

parameters. This procedure is repeated for each training iteration across participating clients. This architectural partitioning enables clients to preprocess local data on-device and to transmit only intermediate feature maps to the server, thereby utilizing distributed compute resources [20].

Despite these advantages, SL incurs excessive communication overhead [9], [21], every forward and backward pass requires round-trip transmission of large dimensional feature maps feature maps, which slow down training efficiency.

B. Communication Compression

1) Biased Compression Techniques: Quantization [12], [22], [23], [24] aims to reduce the bit width of numerical values by transforming them from their continuous forms to discrete counterparts, thus reducing the data storage demands. However, quantization inherently exhibits sensitivity toward outliers, leading to potentially significant compression errors. Methods based on clipping [12] mitigate the impact of these outliers by applying clipping techniques, yet the core issue is that outliers in feature maps are essential and cannot merely be overlooked. Alternatives such as clustering quantization [25] show promise in diminishing these compression errors. However, due to their substantial computational overhead and complexity, these approaches are often impractical for high-frequency iterative training scenarios that demand fast and efficient computation. In addition, sparsification [26] converts dense vectors into their sparse forms. A prevalent technique is the Top-k sparsification [27], which retains only the most significant k values of a vector.

2) Unbiased Compression Techniques: Unbiased random quantization [28] distinguishes itself through random rounding procedures, marking an advancement in quantization that guarantees the unbiased nature of the operation. Similarly, there is also an unbiased variant of sparsification (Rand-k) [29] by enlarging randomly retained values, which guarantees an unbiased sparsification process.

In the learning paradigm of transmission gradients, the application of unbiased compression techniques has been shown to enable DNNs to converge and to realize a convergence rate the same as the uncompressed gradient [28], [29]. However, directly applying unbiased compression to feature maps in SL does not guarantee the generation of unbiased gradients. Additionally, these compression technologies tend to induce substantial compression errors, challenging their applicability for feature map compression in SL.

C. Feature Compression for Split Inference

Feature compression for split inference has recently emerged as an important direction that lies between federated learning and split learning, where intermediate activations are transmitted under bandwidth constraints. Early works focus on supervised feature compression [30], while subsequent studies introduce learnable bottleneck mechanisms such as variational compression [31] and communication-aware feature encoding [32]. Task-oriented approaches further optimize the trade-off between communication cost and task performance based on the information bottleneck principle [33]. Recent work also explores adaptive feature compression and in-orbit annotation for satellite computing [34].

Different from these methods, which rely on additional encoder–decoder architectures or learned compression modules, our approach adopts a lightweight sparsification strategy directly on feature maps, providing a more efficient and model-agnostic solution for split learning.

III. Method

A. Problem Formulation

We consider an SL system consisting of N clients [35], [36] under the wireless environment. Each client $i \in \{1, 2, \ldots, N\}$ is equipped with a local model $\mathcal{F}^c(\cdot)$, parameterized by $\mathbf{w}^c$, which produces intermediate feature maps that serve as the input to the corresponding server-side model $\mathcal{F}^s(\cdot)$, parameterized by $\mathbf{w}^s$. Although all clients share the same parameters $\mathbf{w}^c$ and $\mathbf{w}^s$, we use indexed notation $\mathbf{w}_i^c$ and $\mathbf{w}_i^s$ for clarity in derivation. The loss function is denoted as $\mathcal{L}(\cdot, \cdot)$.

Each client i holds a local dataset $(\mathbf{x}_i, \mathbf{y}_i)$. During forward propagation, the client computes the smashed data $\mathbf{s}_i := \mathcal{F}^c(\mathbf{x}_i; \mathbf{w}_i^c)$ and transmits a compressed version $\hat{\mathbf{s}}_i$ to the server for further processing. For simplicity, the compression is modeled as an additive perturbation: $\hat{\mathbf{s}}_i := \mathbf{s}_i + \epsilon_i$, where ϵ_i denotes the compression error.

At the server side, the received smashed data $\hat{\mathbf{s}}_i$ is fed into the server model, and the loss is computed using the ground-truth label $\mathbf{y}_i$:

$$l := \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(\mathcal{F}^s\left(\hat{\mathbf{s}}_i; \mathbf{w}_i^s\right), \mathbf{y}_i\right). \tag{1}$$

During backward propagation, the server computes gradients $\hat{\mathbf{g}}_i^s = \nabla \mathbf{w}_i^s l$ and aggregates them as $\hat{\mathbf{g}}^s = \frac{1}{N} \sum_{i=1}^{N} \hat{\mathbf{g}}_i^s$. The gradient with respect to $\hat{\mathbf{s}}_i$ is sent back to the client. The client-side gradient is approximated by:

$$\hat{\mathbf{g}}_i^c \approx \nabla_{\mathbf{w}_i^c} \mathbf{s}_i \cdot \nabla_{\hat{\mathbf{s}}_i} l, \tag{2}$$

and aggregated as $\hat{\mathbf{g}}^c = \frac{1}{N} \sum_{i=1}^{N} \hat{\mathbf{g}}_i^c$.

The model parameters are updated using stochastic gradient descent (SGD) with learning rate η:

$$\mathbf{w}^c \leftarrow \mathbf{w}^c - \eta \cdot \hat{\mathbf{g}}^c, \ \mathbf{w}^s \leftarrow \mathbf{w}^s - \eta \cdot \hat{\mathbf{g}}^s. \tag{3}$$

The global optimization objective is:

$$\mathcal{G}\left(\mathbf{w}^c, \mathbf{w}^s\right) := \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(\mathcal{F}^s\left(\mathcal{F}^c\left(\mathbf{x}_i; \mathbf{w}^c\right) + \epsilon_i; \mathbf{w}^s\right), \mathbf{y}_i\right), \tag{4}$$

and the training goal becomes:[2]

$$\left(\mathbf{w}^c \oplus \mathbf{w}^s\right)^* := \arg \min_{\mathbf{w}^c \oplus \mathbf{w}^s} \mathcal{G}\left(\mathbf{w}^c, \mathbf{w}^s\right). \tag{5}$$

In the case of uncompressed transmission (i.e., $\epsilon_i = \mathbf{0}$), we denote the exact gradients as $\mathbf{g}_i^s, \mathbf{g}_i^c$, and their averages as $\mathbf{g}^s, \mathbf{g}^c$ respectively.

[2]In this paper, we use $\mathbf{v}_1 \oplus \mathbf{v}_2$ to represent column vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ concatenated by columns.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

B. Top-K Sparsification

In SL, feature map values with larger magnitudes typically have a greater impact on training. Perturbations to these values lead to more significant changes in gradients, whereas smaller values are often suppressed by ReLU activations or discarded during pooling, rendering their perturbations less important (see Section V-B1 for our experiments on sparsification strategies).

To exploit this property, Top-k sparsification retains only the most informative (i.e., largest) elements in the feature map. This reduces the communication overhead by encoding the positions of the Top-k values using a binary mask $\mathbf{m}$ (requiring n bits for n elements), and transmitting only the k retained floating-point values in vector $\mathbf{v}$. When $k \ll n$, this leads to substantial compression.

However, setting all non-Top-k values to zero introduces significant reconstruction error, which can hinder model convergence. To mitigate this issue while preserving the efficiency of Top-k sparsification, we propose Mask-Encoded Sparsification (MS). This method reduces reconstruction error by encoding the non-Top-k elements with lower precision instead of discarding them entirely, thereby achieving a better balance between communication efficiency and model accuracy.

C. Masked-Encoded Sparsification

Top-k sparsification discards non-Top-k values, leading to significant reconstruction errors. Given a feature map $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n)$, the Top-$k$ operation selects the k largest-magnitude components. Let $\Omega_k(\mathbf{x})$ denote the set of indices corresponding to these components, such that $|\Omega_k(\mathbf{x})| = k$ and for all $i \in \Omega_k(\mathbf{x})$, $j \notin \Omega_k(\mathbf{x})$, $|\mathbf{x}_i| > |\mathbf{x}_j|$. The mask operator $\mathcal{T}_k : \mathbb{R}^n \to \{0,1\}^n$ is defined as:

$$[\mathcal{T}_k(\mathbf{x})]_i = \begin{cases} 1, & \text{if } i \in \Omega_k(\mathbf{x}) \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

During compression, the client computes a binary mask $\mathbf{m} = \mathcal{T}_k(\mathbf{x})$ and extracts the Top-k values into a vector $\mathbf{v} = (\mathbf{x}_{i_1}, \mathbf{x}_{i_2}, \ldots, \mathbf{x}_{i_k})$, where $i_1 < i_2 < \ldots < i_k$ are the indices in $\Omega_k(\mathbf{x})$. The pair $(\mathbf{m}, \mathbf{v})$ is transmitted to the server.

To reconstruct the sparse feature vector $\hat{\mathbf{x}}$, the server uses a scatter operation $\mathcal{S}(\mathbf{v}, \Omega_k(\mathbf{x}))$, which places the elements of $\mathbf{v}$ at the indices where $\mathbf{m}_i = 1$ and sets other entries to zero. $\Omega_k(\mathbf{x}) = \{i \mid \mathbf{m_i} = 1\}$ is in ascending order. Formally,

$$[\mathcal{S}(\mathbf{v}, \Omega_k(\mathbf{x}))]_i = \begin{cases} \mathbf{v}_j, & \text{if } i = \Omega_k(\mathbf{x})[j],\ j \in [1:k] \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

The reconstructed vector is $\hat{\mathbf{x}} = \mathcal{S}(\mathbf{v}, \Omega_k(\mathbf{x}))$.

We propose Mask-Encoded Sparsification (MS), which extends the mask's bit width to b bits to approximate non-Top-k values while preserving Top-k values exactly. The process begins by applying an absolute value operation to the feature map, setting $\mathbf{x} = |\mathbf{x}|$. The Top-$k$ values are identified by their indices in $\Omega_k(\mathbf{x})$, and the minimum Top-k value is denoted $t = \min(\{\mathbf{x}_i \mid i \in \Omega_k(\mathbf{x})\})$. Non-Top-$k$ values, which are in $[0, t]$, are quantized into $2^b - 1$ discrete levels. The mask operator $\mathcal{M}_k : \mathbb{R}^n \to \{0, 1, \ldots, 2^b - 1\}^n$ is defined as:

$$[\mathcal{M}_k(\mathbf{x}, t, b)]_i = \begin{cases} 0b\underbrace{1 \cdots 1}_{b}, & \text{if } i \in \Omega_k(\mathbf{x}) \\ \text{bin}\left(\lfloor \mathbf{x}_i \cdot \frac{2^b - 1}{t} \rfloor\right), & \text{otherwise} \end{cases} \tag{8}$$

Here, the $0b$ prefix represents that it is a binary string and $\text{bin}(\cdot)$ converts an integer to its b-bit binary representation. A sign mask $\mathbf{s} \in \{0,1\}^n$ is computed, where $\mathbf{s}_i = 0$ if $\mathbf{x}_i \geq 0$ and $\mathbf{s}_i = 1$ otherwise. If the feature map is non-negative (e.g., post-ReLU), $\mathbf{s}$ can be omitted.

The client computes $\mathbf{m} = \mathcal{M}_k(\mathbf{x}, t, b)$ and extracts the Top-k values into $\mathbf{v} = (\mathbf{x}_{i_1}, \mathbf{x}_{i_2}, \ldots, \mathbf{x}_{i_k})$, where $i_1 < i_2 < \ldots < i_k$ are the indices in $\Omega_k(\mathbf{x})$. The tuple $(\mathbf{m}, \mathbf{v}, \mathbf{s})$ (or $(\mathbf{m}, \mathbf{v})$ if $\mathbf{s}$ is omitted) is sent to the server.

At the server, the feature vector $\hat{\mathbf{x}}$ is reconstructed using a modified scatter operation $\mathcal{S}'$. Top-k values are restored from $\mathbf{v}$, and non-Top-k values are approximated by scaling the mask values back to $[0, t]$. Formally,

$$[\mathcal{S}'(\mathbf{v}, \mathbf{m}, \mathbf{s}, \Omega_k(\mathbf{x}), t, b)]_i = \begin{cases} (-1)^{\mathbf{s}_i} \cdot \mathbf{v}_j, & \text{if } i = \Omega_k(\mathbf{x})[j],\ j \in [1:k] \\ (-1)^{\mathbf{s}_i} \cdot \mathbf{m}_i \cdot \frac{t}{2^b - 1}, & \text{otherwise} \end{cases} \tag{9}$$

The reconstructed vector is $\hat{\mathbf{x}} = \mathcal{S}'(\mathbf{v}, \mathbf{m}, \mathbf{s}, t, b)$. If $\mathbf{s}$ is omitted, $(-1)^{\mathbf{s}_i}$ is set to 1.

To apply MS in the SL framework, the client first computes the intermediate feature map and constructs a sparse representation by selecting the top-k activations and encoding their positions using a compact binary mask. If the feature map contains negative values, a sign mask is also generated to retain the sign information. The client then transmits the binary mask, the selected Top-k values, and the sign mask to the server. Upon receiving these components, the server reconstructs an approximate feature map and continues the forward propagation using its local model. The complete process is detailed in Algorithm 1.

D. Time Complexity

We analyze the computational efficiency of MS in both the encoding (client-side) and decoding (server-side) stages, which are critical in the SL setting due to frequent iterative communication.

During encoding, Top-k sparsification requires $\mathcal{O}(d \log(k))$ time, primarily due to the Top-k selection from the feature map of length d, where k denotes the number of values to retain. MS introduces an additional mask encoding step with time complexity $\mathcal{O}(d)$, resulting in a total encoding complexity of $\mathcal{O}(d \log(k) + d)$. On the server side, decoding involves reconstructing the sparse feature map based on the received binary mask, value vector, and sign bits. This reconstruction process in MS also runs in linear time, i.e., $\mathcal{O}(d)$, making it efficient for large-scale applications.

In comparison, other compression schemes such as clustering-based quantization [25] may achieve lower compression error, but often incur higher encoding and decoding complexity. Such overhead makes them less suitable for SL, where low-latency

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Mask-Encoded Sparsification in SL.

Input: Client input data $\{\mathbf{d}_i\}_{i=1}^N$
Parameter: Number of Top values k, mask width b, client and server neural network $\mathcal{F}_c, \mathcal{F}_s$

Client: Forward Propagation and Encoding
for each client $i = 1$ to N **in parallel do**
  Compute feature map $\mathbf{x}_i = \mathcal{F}_c(\mathbf{d}_i; w_i^c)$
  if $\mathbf{x}_i$ is non-negative (e.g., post-ReLU) **then**
    Set sign mask $\mathbf{s}_i = (0, \ldots, 0)$
  else
    $s_{i,j} = 0$ if $x_{i,j} \geq 0$, else $s_{i,j} = 1$
    Set $\mathbf{x}_i = |\mathbf{x}_i|$
  end if
  Compute $\Omega_k(\mathbf{x}_i)$ as indices of the k largest values
  Find upper bound of non-Top values: $t_i = \min(\{x_{i,j} \mid j \in \Omega_k(\mathbf{x}_i)\})$
  Compute mask $\mathbf{m}_i = \mathcal{M}_k(\mathbf{x}_i, t_i, b)$
  Compute vector $\mathbf{v}_i = (\mathbf{x}_{i,j_1}, \ldots, \mathbf{x}_{i,j_k})$, where $j_1 < \ldots < j_k \in \Omega_k(\mathbf{x}_i)$
  Send $(\mathbf{m}_i, \mathbf{v}_i, \mathbf{s}_i)$ to the server
end for
Server: Decoding and Forward Propagation
for each client $i = 1$ to N **do**
  Receive $(\mathbf{m}_i, \mathbf{v}_i, \mathbf{s}_i)$
  Find the upper bound $t_i = \min(\mathbf{v}_i)$
  Reconstruct $\Omega_k(\mathbf{x}_i) = \{j \mid m_{i,j} = 2^b - 1\}$
  Reconstruct $\hat{\mathbf{x}}_i = \mathcal{S}'(\mathbf{v}_i, \mathbf{m}_i, \mathbf{s}_i, \Omega_k(\mathbf{x}_i), t_i, b)$
end for
Compute server output $y = \mathcal{F}_s(\{\hat{\mathbf{x}}_i\}_{i=1}^N; w_s)$
Backward Propagation: Compute server gradients g_s and client gradients $\{g_c^i\}_{i=1}^N$, send g_c^i back to each client i for parameter updates

training iterations are essential. Moreover, for the same compression ratio, MS typically selects fewer top elements (i.e., a smaller k) than standard Top-k methods, thereby reducing the higher-order term in the encoding complexity. As a result, despite the additional encoding step, MS can achieve better overall time efficiency than Top-k sparsification in practical SL scenarios.

E. Method Insights

MS appears to be a combination of Top-k sparsification and quantization, but its core novelty lies in being tailored for SL feature map transmission. In SL, Top activations are critical, and sparsification preserves them effectively, whereas directly discarding non-Top values increases reconstruction error. MS addresses this by expanding the mask bit width to encode non-Top values, with quantization as a simple implementation. Because all non-Top values are naturally smaller than the smallest Top value, MS can encode them without being affected by outliers, effectively mitigating the impact of outliers during quantization.

Compared with existing methods that naively combine sparsification and quantization [37], [38], [39], [40], MS fully preserves Top values, efficiently encodes non-Top values, and reduces reconstruction error. This design achieves lower compression error while being naturally aligned with SL feature map requirements.

IV. Feature Map Compression Analysis

In this section, we first analyze the gradients derived from compressed feature maps in Section IV-A, and prove that any compression method generally introduces gradient bias. Despite the biased gradient, in Section IV-B we present a convergence analysis of feature map compression in SL, and establish conditions under which the training still converges to the optimal solution. Additionally, Section IV-C reveals the effect of cutlayer selection in SL, showing that networks become increasingly sensitive to feature map errors when the cutlayer is close to the input of the neural network.

A. Gradient Analysis

In FL, unbiased compression techniques such as random quantization or Rand-k sparsification are often employed to ensure that the aggregated gradients remain unbiased. This property is useful in mitigating long-term error accumulation across communication rounds. However, in SL, even when applying unbiased compression on the feature maps (i.e., $\mathbb{E}(\hat{\mathbf{s}}) = \mathbf{s}$), the gradients computed from these compressed feature maps generally exhibit estimator bias, meaning that the expected compressed gradient differs from the true gradient due to the nonlinearity of the neural network.

Theorem 1 (Compressed Feature Maps Generally Induce Estimator Bias): Even when an unbiased compression method is applied to feature maps in SL (i.e., $\mathbb{E}[\hat{s}] = s$), the expected gradient is generally different from the true gradient:

$$\mathbb{E}\left(\hat{\mathbf{g}}^c \oplus \hat{\mathbf{g}}^s\right) \neq \mathbf{g}^c \oplus \mathbf{g}^s \tag{10}$$

Proof: Please refer to Appendix A-A (available online) for the proof of Theorem 1. ■

This result implies that compressed feature map generally introduces biased gradients in SL, regardless of whether the compression method itself is unbiased, due to the nonlinear nature of neural network operations. Therefore, it is necessary to analyze the convergence under biased gradients, which we address in the following section.

B. Convergence Analysis

Building on the previous analysis of gradient bias induced by feature map compression, this section establishes the impact of feature map compression error on the convergence of SL. We first introduce the commonly used assumption in optimization theory of machine learning [41], [42], [43].

Assumption 1 (Lipschitz Smoothness): There exists a finite positive constant L such that for all $\mathbf{w}_1^c, \mathbf{w}_2^c$ in the client-side model and $\mathbf{w}_1^s, \mathbf{w}_2^s$ in the server-side model, the Lipschitz condition is satisfied as follows:

$$\left\|\nabla\mathcal{G}\left(\mathbf{w}_1^c, \mathbf{w}_1^s\right) - \nabla\mathcal{G}\left(\mathbf{w}_2^c, \mathbf{w}_2^s\right)\right\| \leq L\left\|\mathbf{w}_1^c \oplus \mathbf{w}_1^s - \mathbf{w}_2^c \oplus \mathbf{w}_2^s\right\|. \tag{11}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

Assumption 2 (Unbiased Gradients): For every mini-batch, the stochastic gradients of the client-side and server-side models are unbiased estimators of the true gradient:

$$\mathbb{E}\left(\mathbf{g}^c \oplus \mathbf{g}^s\right) = \nabla \mathcal{G}\left(\mathbf{w}^c, \mathbf{w}^s\right). \tag{12}$$

Assumption 3 (Bounded Variance): The variances of the stochastic gradients in the client-side and server-side models are bounded by a finite positive constant σ^2:

$$\mathbb{E}\left\|\nabla \mathcal{G}\left(\mathbf{w}^c, \mathbf{w}^s\right) - \mathbf{g}^c \oplus \mathbf{g}^s\right\|^2 \leq \sigma^2. \tag{13}$$

Given Assumptions 1–3, Let U_c denote the upper bound of the Frobenius norm of the derivative of the cutlayer activations with respect to the client-side parameters. Let U_s denote the upper bound of the Frobenius norm of the derivative of the server-side gradient with respect to the server-side input feature map. We also define the average compression error as $E := \frac{1}{N}\sum_{i=1}^{N}\|\epsilon_i\|$, where ϵ_i is the compression error for the i-th client. We establish the following lemma to bound the gradient errors introduced by feature map compression on both the client and server sides.

Lemma 2 (Bounded Gradient Error): Under the above assumptions and definitions, the discrepancies between the exact and compressed gradients are bounded as follows:

$$\|\hat{\mathbf{g}}^s - \mathbf{g}^s\| \leq U_s E, \ \|\hat{\mathbf{g}}^c - \mathbf{g}^c\| \leq U_c U_s E. \tag{14}$$

Proof: Please refer to Appendix A-B (available online) for the proof of Lemma 2 ■.

Let the subscript t denote the iteration index. For clarity, let $\mathbf{W}_t := \mathbf{w}_t^c \oplus \mathbf{w}_t^s$ denote the concatenation of the client-side and server-side model parameters, and $\mathbf{G}_t := \mathbf{g}_t^c \oplus \mathbf{g}_t^s$ denote the concatenation of their corresponding gradients. Define the accumulated average compression error as $\varepsilon^2 := (1 + U_c^2)U_s^2 \cdot \frac{1}{T}\sum_{t=1}^{T} E_t^2$, where E_t denotes the average compression error at iteration t. Also, let $\gamma := \mathbb{E}[\mathcal{G}(\mathbf{w}_1^c, \mathbf{w}_1^s) - \mathcal{G}(\mathbf{w}_{T+1}^c, \mathbf{w}_{T+1}^s)]$. Building on Assumptions 1–3 and Lemma 2, we derive the convergence rate of SL under compressed feature maps.

Theorem 3 (Convergence in SL): Under Assumptions 1–3, let the learning rate $\eta = \sqrt{\gamma/TL(\sigma^2 + \varepsilon^2)}$. The convergence of SL with compressed feature maps can be characterized under two cases:

- *Case 1:* If the error gradient satisfies the non-negative correlation condition

$$\mathbb{E}\left\langle \nabla \mathcal{G}\left(\mathbf{w}_t^c, \mathbf{w}_t^s\right), \hat{\mathbf{G}}_t - \mathbf{G}_t \right\rangle \geq 0. \tag{15}$$

  Let $\eta \in (0, \frac{1}{2L})$, then

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left\|\nabla \mathcal{G}\left(\mathbf{w}_t^c, \mathbf{w}_t^s\right)\right\|^2 \leq 4\sqrt{\frac{\gamma L(\sigma^2 + \varepsilon^2)}{T}}. \tag{16}$$

- *Case 2:* If the non-negative correlation condition may not hold. Let $\eta \in (0, \frac{1}{4L})$, we have

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left\|\nabla \mathcal{G}\left(\mathbf{w}_t^c, \mathbf{w}_t^s\right)\right\|^2 \leq 8\sqrt{\frac{\gamma L(\sigma^2 + \varepsilon^2)}{T}} + 2\varepsilon^2. \tag{17}$$

Proof: Please refer to Appendix A-C (available online) for the proof of Theorem 3. ■

Remark 1: The iterative nature of SL leads to significant communication overhead during feature map transmission. While Theorem 1 provides a theoretical characterization of estimator bias induced by feature map compression. In practice, although this bias may affect convergence behavior, Theorem 3 characterizes convergence under estimator bias in two cases, the first under an idealized condition and the second under a more general condition, and both cases indicate that lower compression error leads to better convergence.

C. Analysis on the Effect of the Cutlayer Position

To investigate the impact of cutlayer selection on neural networks, we conduct the analysis to a broad class of neural network layers. Let n represent the number of layers in the neural network, denote each layer as a differentiable mapping $\Phi_i : \mathbb{R}^{m_{i-1}} \to \mathbb{R}^{m_i}$ followed by an element-wise activation $\phi(\cdot)$, so that the output of the i-th layer can be written as

$$\mathbf{x}_i = \phi\left(\Phi_i(\mathbf{x}_{i-1})\right), i = 1, 2, \ldots, n, \tag{18}$$

with the original input $\mathbf{x}_0 \in \mathbb{R}^{m_0}$. Here, each Φ_i represents a general layer mapping that is differentiable, continuous, and Lipschitz continuous with constant L_i. Using $\mathcal{L}(\cdot)$ to represent the loss function, the final output of the neural network is a scalar $y = \mathcal{L}(\mathbf{x}_n)$.

Theorem 4 (The impace of cutlayer on SL): When the cutlayer is shallower, the final output error tends to be larger due to the cumulative propagation of the initial compression error through more subsequent layers. Moreover, this amplified output error will correspondingly increase the gradient errors during backpropagation.

Proof: Please refer to Appendix A-D (available online) for the proof of Theorem 4. ■

This result indicates that when the cutlayer is shallower (i.e., when k is smaller), the final output error tends to be larger due to the cumulative propagation of the initial compression error through more subsequent layers. Moreover, this amplified output error will correspondingly increase the gradient errors during backpropagation.

Remark 2: Theorem 4 demonstrates the important role of cutlayer selection. Shallow cutlayers amplify both forward and backward propagation errors. Deep cutlayers reduce propagation errors but impose higher computational and memory burdens on edge devices. This trade-off necessitates careful architectural optimization based on system constraints and accuracy requirements. Developing a systematic method for optimal cutlayer selection remains an interesting direction for future research.

V. Experiments

A. Experimental Setup

1) System Model: We consider an SL system consisting of 10 clients and a server, operating in a wireless environment with the server at the center. The network covers a circular

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SYSTEM PARAMETERS

Description	Value
Bandwidth of sub-channel	$B = 10$MHz
Path loss exponent	$\alpha = 3.5$
Rayleigh fading parameter	o_i exponential distribution (mean 1)
Transmission power of the client	$P^c = 5$W
Noise power	$N_0 = -174$dBm/Hz

area with a radius of 500 m, where the server is located at the center and the clients are uniformly distributed in the area. The distance between the server and clients is randomly selected from 100 m to 500 m. We consider all sub-carriers grouped into 5 sub-channels. The values of other system parameters are listed in Table I. The wireless environment is similar to previous work [5], [44], [45].

2) Baseline Methods: We evaluate our proposed MS against five compression approaches to provide a comprehensive assessment of its effectiveness. Specifically, we compare MS with the following baselines: Vanilla Quantization (**VQ**) [22], [23], [28], which serves as the standard uniform quantization baseline by discretizing values into uniformly spaced intervals; Quantile Quantization (**QQ**) [46], a non-uniform quantization method leveraging percentile-based thresholds derived from the data distribution; FpQuantization (**FPQ**) [47], utilizing floating-point format to achieve flexible and precision-adaptive quantization; Vanilla Sparsification (**VS**) [14], [27], [48], the classical magnitude-based Top-k sparsification method that retains the k largest-magnitude elements; and RandTopk Sparsification (**RTS**) [21], which randomly selects elements from a Top-k subset to introduce stochasticity into sparsification. These benchmarks collectively enable a thorough comparison of MS in terms of compression effectiveness, algorithm efficiency, convergence behavior, and impact on model accuracy.

3) Model Architectures: Our evaluation covers both computer vision (CV) and natural language processing (NLP) scenarios. In CV tasks, we use VGG16 [49] on CIFAR10, a 16-layer network composed of thirteen 3×3 convolutional layers grouped into five blocks, each followed by a max-pooling layer, and capped by three fully connected layers. ReLU activations are applied throughout in VGG16. The network is cut after convolutional layers 1, 7, and 13. We adopt ResNet18 [50] on CIFAR100, which begins with a 7×7 convolution and pooling stage and then stacks eight residual blocks (two 3×3 convolutions per block) with identity shortcuts. ReLU is again used after every convolution in ResNet18. We split the model after layers 1, 9, and 17, each marking the end of a residual block. In NLP tasks, for language modelling on WikiText-2 [51], we evaluate two decoder-only Transformer models. Qwen3-1.7B [52] comprises 28 Transformer blocks with multi-head self-attention and gated feed-forward networks, totaling 1.7 billion parameters; cutlayers are placed after decoder blocks 1, 14, and 27. Llama3.2-1B [53] is a lighter 1 billion parameter model with 16 Transformer blocks that incorporate rotary positional encoding and an optimized feed-forward design. We split it after blocks 1, 8, and 15.

4) Compression Configuration: Experiments on compression parameters are presented in Section V-B3. The default compression configuration is as follows:

The compression configurations are carefully designed to ensure fair comparison across methods. For CV tasks where ReLU activation ensures non-negative features, we find that 2-bit masking combined with 99% sparsification achieves a favorable trade-off between compression efficiency and model performance, resulting in 92.75% overall compression. Comparable compression rates are obtained with VS and RTS at 95.875% sparsification. The quantization baselines VQ and QQ employ 3-bit quantization (90.625% compression ratio), while FPQ uses a 2-bit exponent with 1-bit mantissa configuration (maintaining the same 90.625% compression ratio).

For NLP tasks with SiLU-activated features, Our experiments indicate that 3-bit (including 1 sign bit) masking with 99% sparsification achieve the best balance between compression and accuracy, achieving 89.625% compression. VS and RTS achieve equivalent compression at 92.75% sparsification. The quantization baselines maintain consistent configurations with the vision experiments: VQ and QQ at 3-bit (90.625% compression ratio) and FPQ with 2-bit exponent plus 1-bit mantissa (90.625% compression ratio).

5) Training Protocol: All experiments follow a unified training protocol. For CV tasks, each model is trained for 200 epochs with an initial learning rate of 0.1 that is subsequently reduced according to a predefined decay schedule. For NLP tasks, the pre-trained Transformer models are carefully fine-tuned for 3 epochs using a learning rate of 1×10^{-4}, a low value that preserves their existing knowledge while allowing domain adaptation.

B. Experimental Results

1) Effect of Sparsification Strategies: To justify the design choice of building MS upon Top-k sparsification, we compare different selection strategies, including Top-k, Rand-k, and Bottom-k, under various sparsification ratios and cutlayer settings. The results for CV and NLP tasks are reported in Table II and Table III, respectively.

As shown in Table II, Top-k consistently outperforms both Rand-k and Bottom-k across all models, cutlayers, and sparsification ratios. For example, on VGG16 with cutlayer at Layer 1 and $k = 1\%$, Top-k achieves 90.95% accuracy, significantly higher than Rand-k (66.33%) and Bottom-k (10.00%). Similar trends are observed across deeper cutlayers and on ResNet18. Notably, Bottom-k almost completely fails in all settings, indicating that small-magnitude activations carry limited useful information for downstream tasks. Rand-k performs better than Bottom-k, but still shows a clear performance gap compared to Top-k, especially at low sparsification ratios (e.g., $k = 0.1\%$).

Table III further validates these observations on large language models. Overall, Top-k achieves better or comparable perplexity in most configurations, and demonstrates significantly higher stability compared to other strategies. In contrast, Rand-k frequently leads to severe performance degradation or even

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ACCURACY (%) UNDER DIFFERENT SPARSIFICATION RATIOS ($k \in 0.1\%, 1\%, 5\%$) AND SELECTION STRATEGIES (TOP-K, RANDOM-K, BOTTOM-K) ACROSS DIFFERENT CUTLAYERS FOR VGG16 AND RESNET18

Models	Cutlayers	k	Accuracy under Different Strategies (%)		
			Top-k	Rand-k	Bottom-k
VGG16	Layer 1	0.1%	**61.36**	41.56	10.00
		1%	**90.95**	66.33	10.00
		5%	**93.60**	84.62	10.00
	Layer 7	0.1%	**90.74**	23.45	10.00
		1%	**93.05**	90.88	10.00
		5%	**93.53**	93.08	10.00
	Layer 13	0.1%	**27.06**	14.56	10.00
		1%	**93.93**	29.07	10.00
		5%	**94.20**	92.77	10.00
ResNet18	Layer 1	0.1%	**16.06**	9.78	1.00
		1%	**60.71**	36.26	1.00
		5%	**74.98**	60.24	1.00
	Layer 9	0.1%	**52.43**	12.17	1.00
		1%	**71.57**	58.74	1.00
		5%	**75.66**	69.54	1.00
	Layer 13	0.1%	**67.63**	24.72	1.00
		1%	**75.68**	74.87	1.00
		5%	**77.52**	76.48	1.00

TABLE III
PPL UNDER DIFFERENT SPARSIFICATION RATIOS ($k \in 10\%, 20\%, 30\%$) AND SELECTION STRATEGIES (TOP-K, RANDOM-K, BOTTOM-K) ACROSS DIFFERENT CUTLAYERS FOR QWEN3-1.7B AND LLAMA3.2-1B

Models	Cutlayers	k	PPL under Different Strategies		
			Top-k	Rand-k	Bottom-k
Qwen3-1.7B	Layer 1	10%	INF	INF	INF
		20%	29.74	32909.42	**21.28**
		30%	22.47	23895.1	**19.62**
	Layer 14	10%	**19.76**	INF	22.74
		20%	**18.25**	184.18	21.27
		30%	**17.85**	68.59	20.52
	Layer 27	10%	INF	INF	INF
		20%	**48.07**	50212.85	INF
		30%	1553.64	7573.72	**25.06**
Llama3.2-1B	Layer 1	10%	**15.60**	6035.76	323.69
		20%	**13.94**	3926.92	14.51
		30%	**13.49**	2638.04	13.97
	Layer 8	10%	**16.04**	4707.26	17.87
		20%	14.83	2781.19	**14.74**
		30%	**14.15**	107.90	14.26
	Layer 15	10%	1194.84	2893.69	**23.35**
		20%	**16.99**	2251.8	18.97
		30%	**15.84**	814.11	17.51

divergence (indicated by INF values), particularly under aggressive sparsification. Bottom-k exhibits more stable behavior than Rand-k in some cases, but generally results in inferior perplexity due to the loss of informative high-magnitude activations. For example, on Qwen3-1.7B at Layer 14 with $k = 20\%$, Top-k achieves a perplexity of 18.25, while Rand-k degrades to 184.18, and Bottom-k reaches 21.27. Although Top-k is not strictly optimal in every setting, it consistently provides a good trade-off between performance and robustness across different models, cutlayers, and sparsification levels.

In our experiments, the Top-k selection strategy is generally effective across different models and tasks. In CV tasks with ReLU activations and pooling layers (Table II), Top-k consistently outperforms Rand-k and Bottom-k, with Bottom-k failing to converge and yielding near-random accuracy, indicating that low-magnitude features contribute minimally. In NLP tasks with SiLU activations and no pooling layers (Table III), Top-k achieves the best performance in most cases, while Rand-k leads to higher perplexity. Bottom-k occasionally achieves optimal perplexity in some specific layers and configurations, showing that low-magnitude features can sometimes be useful. Overall, these results validate Top-k as an effective default strategy, and MS further improves robustness by preserving non-top values through low-bit quantization, retaining potentially useful information in scenarios where low-magnitude features contribute to performance.

Based on these observations, Top-k provides the most effective sparsification strategy and serves as a strong foundation for feature map compression. However, directly discarding non-Top-k values still introduces substantial reconstruction error. This motivates our proposed Mask-Encoded Sparsification, which retains the advantages of Top-k while further reducing the error of discarded elements through low-bit encoding.

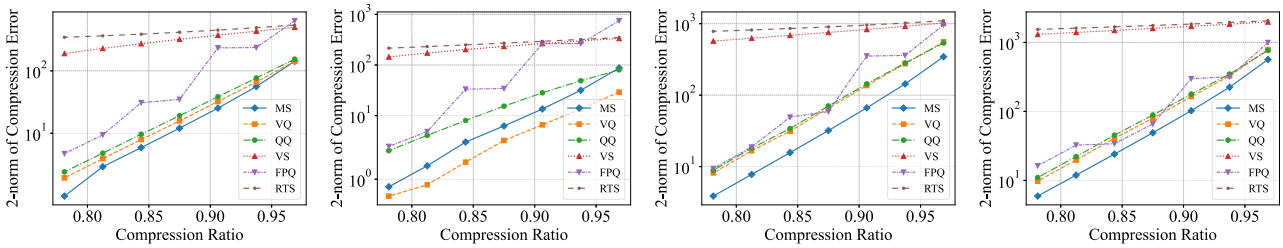
2) Error Comparison Experiment: We conduct a comprehensive evaluation of compression errors across different data distributions to systematically compare the performance of the six compression algorithms: MS, VQ, QQ, VS, FPQ, and RTS. The analysis considers four fundamental probability distributions: normal, uniform, exponential, and gamma. To ensure statistical reliability, all results are averaged over 10 independent sampling trials.

The experimental results are presented in two scenarios. Fig. 2 shows results for ReLU-processed non-negative data, where MS benefits from 1-bit mask savings, while Fig. 3 displays results for raw data distributions. For ReLU-processed data, MS demonstrates superior compression performance across all distributions except uniform. In the uniform distribution case, VQ achieves the theoretically optimal performance due to its design assumptions. However, this advantage does not extend to other distributions, where MS shows significant improvements. In the raw data evaluation. QQ achieves the lowest error in normal distributions due to its design assumptions, while VQ maintains its advantage in uniform distributions. MS shows competitive performance in exponential and gamma distributions. The sparsification-based methods VS and RTS exhibit consistently higher errors across all distributions.

The experimental results demonstrate that the MS algorithm achieves superior compression accuracy across most scenarios. Notably, when processing ReLU-activated non-negative data, MS outperforms the QQ algorithm that assumes a normal distribution. This performance advantage aligns with the theoretical guarantees provided in Theorem 3, confirming MS as the more effective approach for compressing SL feature maps to ensure better convergence.

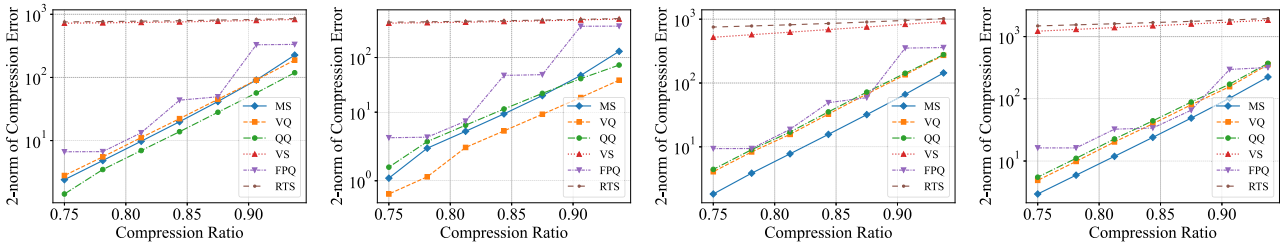
3) Impact of Bit-Width and Sparsification Ratio: To systematically evaluate the impact of bit-width b and sparsification ratio k, we conduct extensive experiments across different tasks,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.



(a) $X \sim \text{Normal}(0,1)$ (b) $X \sim \text{Uniform}(-1,1)$ (c) $X \sim \text{Exp}(1)$ (d) $X \sim \text{Gamma}(2,1)$

Fig. 2. Average compression error versus compression ratio for MS, VQ, QQ, VS, FPR, and RTS methods under four input distributions after ReLU activation. (a) Normally distributed input $X \sim \text{Normal}(0,1)$; (b) Uniformly distributed input $X \sim \text{Uniform}(-1,1)$; (c) Exponentially distributed input $X \sim \text{Exp}(1)$; (d) Gamma distributed input $X \sim \text{Gamma}(2,1)$. All inputs were processed through ReLU to simulate feature map transformations in neural networks.



(a) $X \sim \text{Normal}(0,1)$ (b) $X \sim \text{Uniform}(-1,1)$ (c) $X \sim \text{Exp}(1)$ (d) $X \sim \text{Gamma}(2,1)$

Fig. 3. Average compression error versus compression ratio for MS, VQ, QQ, VS, FPR, and RTS methods under four input distributions: (a) Normal distribution $X \sim \text{Normal}(0,1)$; (b) Uniform distribution $X \sim \text{Uniform}(-1,1)$; (c) Exponential distribution $X \sim \text{Exp}(1)$; (d) Gamma distribution $X \sim \text{Gamma}(2,1)$. Results show raw input performance without ReLU transformation.

TABLE IV
ACCURACY (%) UNDER DIFFERENT BIT-WIDTHS ($b \in \{2,3,4\}$) AND SPARSIFICATION RATIOS ($k \in \{0.1\%, 1\%, 5\%\}$). RESULTS ARE REPORTED AT DIFFERENT CUTLAYERS FOR VGG16 TRAINED ON CIFAR10 AND RESNET18 TRAINED ON CIFAR100. "NOCOMP" DENOTES THE BASELINE WITHOUT FEATURE MAP COMPRESSION.

Models	Cutlayers	NoComp	Accuracy under Different Configurations (%)								
			$b=2$			$b=3$			$b=4$		
			$k=0.1\%$	$k=1\%$	$k=5\%$	$k=0.1\%$	$k=1\%$	$k=5\%$	$k=0.1\%$	$k=1\%$	$k=5\%$
VGG16	Layer 1	94.03	93.53	94.00	93.94	93.32	93.74	94.02	93.84	94.09	93.09
	Layer 7		93.97	94.20	93.88	94.01	93.99	93.95	93.93	93.99	94.19
	Layer 13		93.69	93.85	93.98	94.20	93.84	93.86	93.96	93.91	94.03
ResNet18	Layer 1	77.11	75.53	76.23	76.78	76.32	77.07	76.75	77.51	77.15	76.73
	Layer 9		76.83	77.12	76.87	76.97	77.33	77.02	77.07	77.15	77.19
	Layer 17		77.12	77.27	77.52	77.17	77.17	77.35	77.27	76.89	76.94

model architectures, and cutlayer positions. Specifically, for CV tasks with ReLU-activated non-negative feature maps, we set $b \in \{2,3,4\}$ and $k \in \{0.1\%, 1\%, 5\%\}$. For NLP tasks, where activations (e.g., SiLU) produce signed values, an additional sign bit is required, and thus $b \in \{3,4,5\}$ with the same sparsification settings. The results are summarized in Table IV and Table V. Overall, increasing the bit-width does not lead to significant performance improvements across either CV or NLP tasks. For example, in VGG16 and ResNet18, the accuracy differences between $b=2$, $b=3$, and $b=4$ remain within a very small margin. Similar trends are observed in language models, where increasing b beyond the minimum required value provides negligible gains in perplexity. This indicates that the dominant source of reconstruction fidelity comes from the Top-k values rather than the precision of the encoded mask.

From a system perspective, this suggests that low-bit encoding is sufficient. For non-negative feature maps (e.g., CV models with ReLU), $b=2$ already provides an effective trade-off between compression and accuracy. For signed feature maps (e.g., NLP models), one additional bit is required to encode the sign, making $b=3$ a practical choice. Further increasing the bit-width introduces additional communication overhead without proportional performance benefits. Regarding sparsification, reducing k from 1% to 0.1% can further decrease communication cost while maintaining acceptable performance in some cases. However, the additional compression gain is limited. For instance, assuming 32-bit floating-point representation, using 2-bit encoding with $k=1\%$ results in an average communication cost of approximately 2.32 bits per element, while $k=0.1\%$ reduces it to 2.032 bits. This marginal improvement suggests diminishing

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PPL UNDER DIFFERENT BIT-WIDTHS ($b \in \{3, 4, 5\}$) AND SPARSIFICATION RATIOS ($k \in \{0.1\%, 1\%, 5\%\}$). RESULTS ARE REPORTED AT DIFFERENT CUTLAYERS FOR QWEN3-1.7B AND LLAMA3.2-1B, BOTH FINE-TUNED ON WIKITEXT. "NOCOMP" DENOTES THE BASELINE WITHOUT FEATURE MAP COMPRESSION.

Models	Cutlayers	NoComp	PPL under Different Configurations								
			$b=3$			$b=4$			$b=5$		
			$k=0.1\%$	$k=1\%$	$k=5\%$	$k=0.1\%$	$k=1\%$	$k=5\%$	$k=0.1\%$	$k=1\%$	$k=5\%$
Qwen3-1.7B	Layer 1	15.08	16.02	15.22	15.12	15.30	15.14	15.14	15.14	15.14	15.11
	Layer 14		21.41	15.37	15.18	15.81	15.17	15.13	15.27	15.12	15.12
	Layer 27		28.43	15.68	15.41	18.74	15.20	15.17	16.18	15.14	15.11
Llama3.2-1B	Layer 1	11.13	12.36	11.43	11.20	11.33	11.16	11.11	11.15	11.11	11.09
	Layer 8		89.27	11.26	11.15	11.59	11.12	11.10	11.20	11.09	11.09
	Layer 15		12.84	11.56	11.31	11.53	11.18	11.12	11.21	11.11	11.10

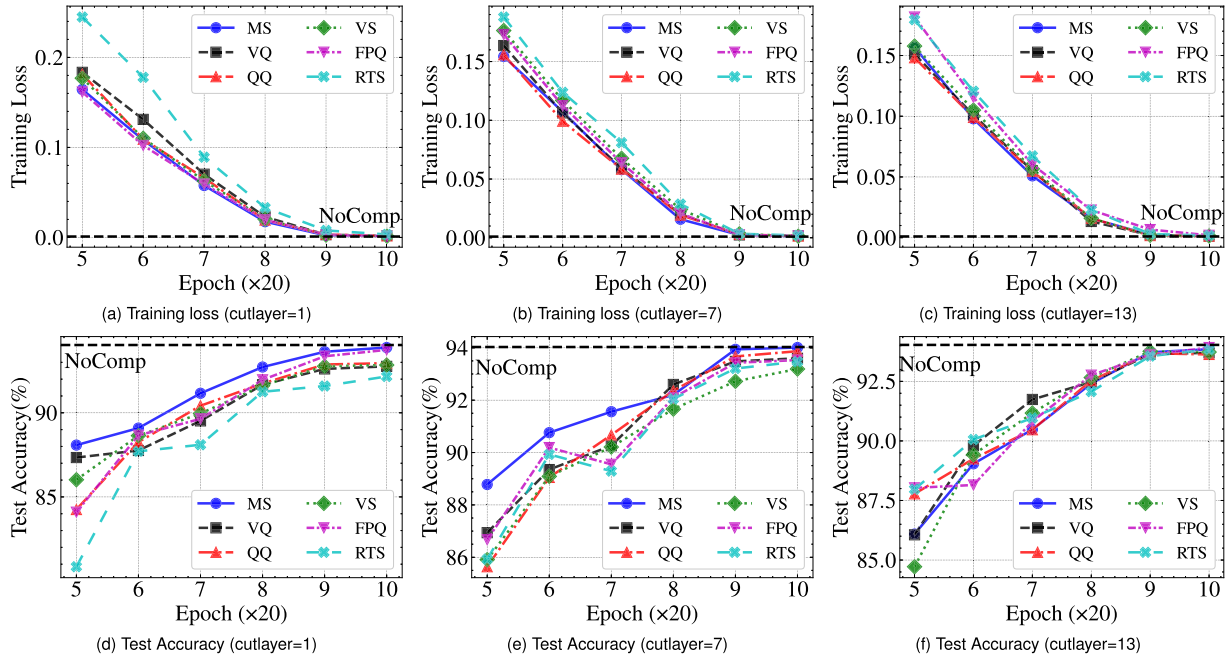


Fig. 4. The training loss and test accuracy of VGG16 on CIFAR10 with different cutlayers: (a)–(c) training loss curves for cutlayer positions at 1, 7, and 13 respectively; (d)–(f) corresponding test accuracy curves for the same cutlayer configurations, "NoComp" denotes the baseline without feature map compression.

returns when aggressively reducing k, especially considering potential performance degradation in certain tasks.

Based on these observations, we adopt a unified configuration in subsequent experiments: 2-bit with $k = 1\%$ for non-negative feature maps, and 3-bit with $k = 1\%$ for signed feature maps. This setting achieves a favorable balance between communication efficiency and model performance across all evaluated scenarios.

4) Computing Efficiency: We also compare the encoding and decoding times of all algorithms on 1×10^7 dimensional data. Fig. 5 presents the encoding and decoding time comparison of six compression algorithms under varying compression ratios.

In the encoding phase (Fig. 5(a)), MS consistently achieves lower encoding time compared to VS and RTS. This is because, under the same compression ratio, MS uses a smaller value of k, leading to reduced computational cost. As the compression ratio decreases, the encoding time of VS and RTS increases steadily, which aligns with their time complexity of $O(d\log(k))$, where larger k values result in greater overhead. In contrast, VQ and FPQ exhibit nearly constant encoding time across different compression ratios, since their complexity remains $O(d)$. QQ shows an increasing trend in encoding time as the compression ratio decreases, due to the overhead introduced by quantile computation.

In the decoding phase (Fig. 5(b)), MS achieves the fastest decoding time in all settings. Most algorithms exhibit comparable decoding efficiency, except for FPQ, whose significantly higher cost stems from its reliance on computationally expensive exponential operations during decoding.

These results demonstrate the practicality of MS, particularly in the encoding phase, which is typically performed on resource-constrained devices. Compared to sparsification-based methods, MS significantly improves execution efficiency.

5) Communication Efficiency: The experimental results in Table VI demonstrate the total communication times over the entire training process for uncompressed transmission and various compression methods applied to VGG16 and ResNet18 models under different cutlayers and target accuracies. The values in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
TOTAL COMMUNICATION TIME OF VGG16 AND RESNET18 UNDER DIFFERENT CUTLAYERS AND ACCURACY TARGETS. RESULTS ARE REPORTED FOR UNCOMPRESSED TRANSMISSION (NO COMP) AND VARIOUS COMPRESSION METHODS (MS, VQ, QQ, VS, FPQ, RTS). VALUES IN PARENTHESES DENOTE THE CORRESPONDING SPEEDUP RELATIVE TO NO COMPRESSION.

Models	Cutlayers	Targets	Communication Time (minutes, with speedup in parentheses)						
			No comp	MS	VQ	QQ	VS	FPQ	RTS
VGG16	layer 1	80%	3397.8	**246.3 (13.79)**	278.7 (12.19)	358.4 (9.48)	338.7 (10.03)	278.7 (12.19)	431.1 (7.88)
		90%	12316.9	**1016.1 (12.12)**	2229.8 (5.52)	1433.4 (8.59)	1108.5 (11.11)	1712.2 (7.19)	1755.2 (7.02)
	layer 7	80%	212.4	**13.5 (15.76)**	17.4 (12.19)	22.4 (9.48)	19.2 (11.03)	22.4 (9.48)	26.9 (7.88)
		90%	769.8	**55.8 (13.79)**	77.1 (9.98)	59.7 (12.89)	86.6 (8.89)	69.7 (11.05)	78.9 (9.76)
	layer 13	80%	26.5	**2.6 (10.03)**	2.8 (9.48)	3.4 (7.76)	3.6 (7.36)	3.7 (7.11)	3.4 (7.88)
		90%	96.2	**6.3 (15.38)**	9.0 (10.67)	8.1 (11.9)	7.9 (12.12)	10.9 (8.84)	11.3 (8.51)
ResNet18	layer 1	80%	13166.3	**1293.3 (10.18)**	2428.9 (5.42)	2428.9 (5.42)	1878.3 (7.01)	1513.1 (8.7)	1878.3 (7.01)
		90%	25908.0	**1878.3 (13.79)**	2428.9 (10.67)	2428.9 (10.67)	1878.3 (13.79)	2428.9 (10.67)	3818.2 (6.79)
	layer 9	80%	6583.2	**508.1 (12.96)**	617.2 (10.67)	696.8 (9.45)	846.8 (7.77)	577.4 (11.4)	939.2 (7.01)
		90%	12954.0	**939.2 (13.79)**	1214.4 (10.67)	1214.4 (10.67)	939.2 (13.79)	1214.4 (10.67)	939.2 (13.79)
	layer 17	80%	1645.8	**77.0 (21.38)**	189.1 (8.7)	124.4 (13.23)	103.9 (15.84)	199.1 (8.27)	100.1 (16.45)
		90%	3238.5	**234.8 (13.79)**	303.6 (10.67)	303.6 (10.67)	234.8 (13.79)	303.6 (10.67)	234.8 (13.79)

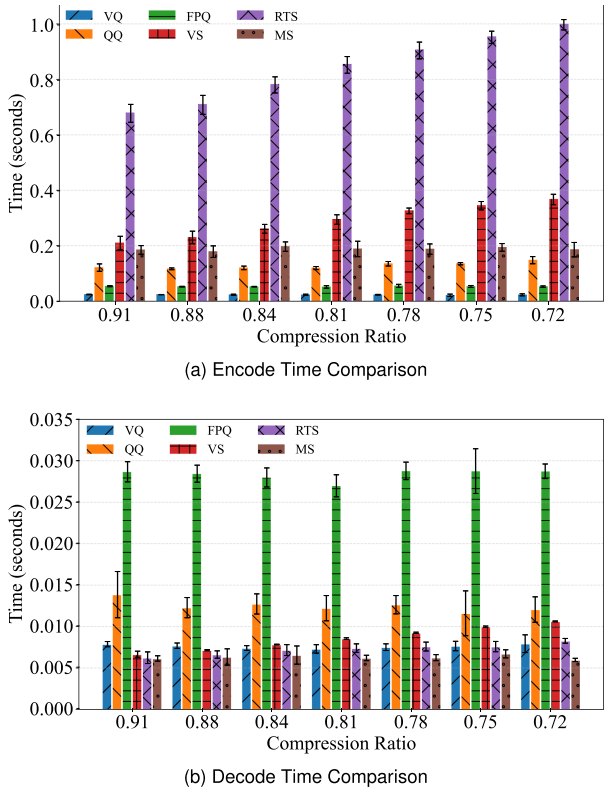


Fig. 5. Comparison of encoding and decoding time across different compression algorithms under varying compression ratios. Each bar represents the average time for encoding or decoding a single vector, with error bars indicating the standard deviation. The compression ratio decreases from left to right.

parentheses represent the speedup relative to no compression. Across all configurations in the simulation environment, our MS method consistently achieves the highest communication efficiency, with the lowest communication times and the most significant speedups. For instance, on VGG16 at layer 13 with 90% target accuracy, MS reduces the time to 6.3 minutes (a 15.38× speedup), outperforming alternatives like VQ (9.0 minutes, 10.67×) and QQ (8.1 minutes, 11.9×). Similar trends hold for ResNet18, where MS at layer 17 with 80% accuracy yields 77.0 minutes (21.38×), surpassing VQ (189.1 minutes, 8.7×) and other methods in overall efficiency. This highlights the superiority of MS in minimizing communication overhead while maintaining high accuracy. The enhancement in communication efficiency directly translates to a substantial acceleration of the SL.

6) CV Tasks: The training dynamics, including loss curves and test accuracy throughout the optimization process, are documented in Figs. 4 and 6. The experimental results demonstrate that MS consistently achieves model performance that closely approximates the uncompressed baseline. Notably, while FPQ shows comparable accuracy preservation to MS, its compression capability is fundamentally constrained by the discrete nature of bit-width configurations. Specifically, FPQ is limited to a 90.625% compression ratio (using 2-bit exponent and 1-bit mantissa) in our experiments, whereas MS achieves a superior 92.75% compression through its continuous sparsity adjustment mechanism. The performance comparison reveals that MS provides a better trade-off between compression efficiency and model accuracy, as it can flexibly continuously adjust compression rates compared to FPQ's discrete compression options.

In addition, these results validate our theoretical analysis presented earlier in Theorem 4, the impact of errors introduced in intermediate features diminishes as the cutlayer moves deeper in the network. This phenomenon is particularly evident in the ResNet18 experiments on CIFAR-100. When the cutlayer is positioned at layer 1 (Fig. 6(d)), although MS achieves the highest test accuracy among the compared methods, there remains a gap from the baseline. This gap decreases as the cutlayer moves deeper while maintaining the same compression rate, confirming our theoretical prediction. However, we emphasize that simply moving the cutlayer deeper is not advisable despite the improved accuracy. Deeper cutlayers significantly increase both storage requirements and computational overhead for clients, which contradicts the fundamental design principles of SL. Therefore, the cutlayer presents a crucial trade-off that must balance model performance against client resource constraints.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

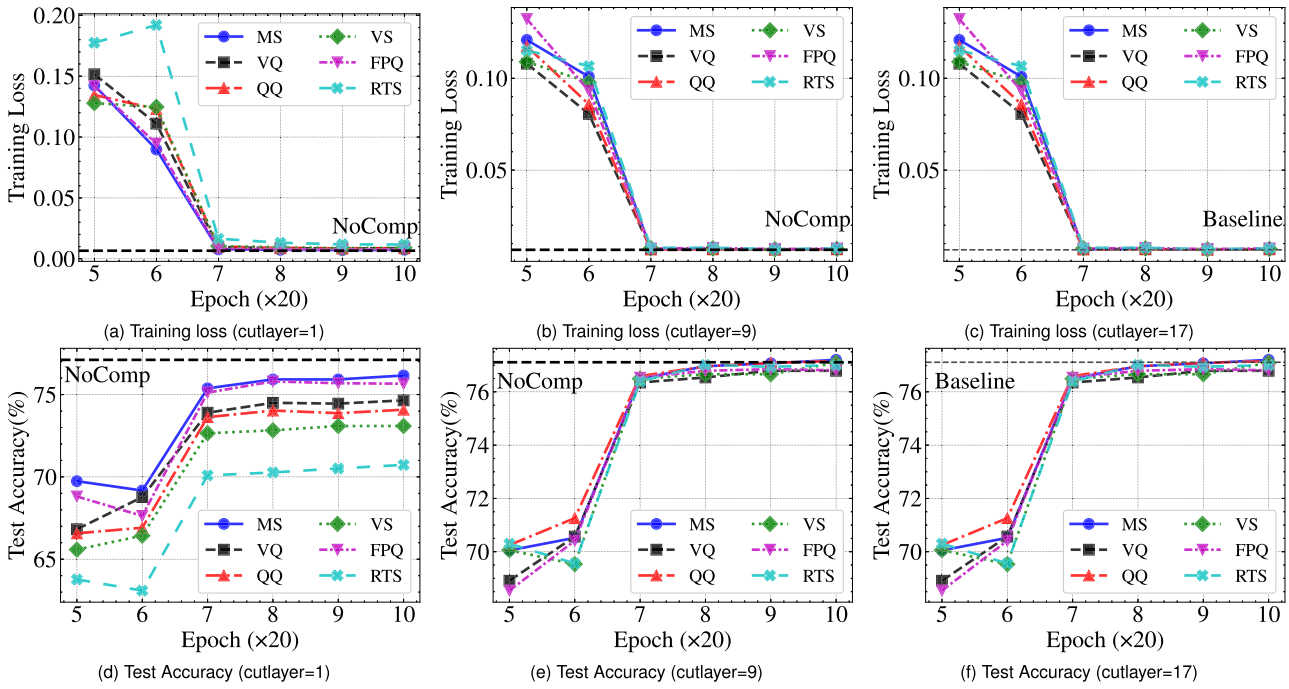


Fig. 6. The training loss and test accuracy of ResNet18 on CIFAR100 with different cutlayers: (a)–(c) training loss curves for cutlayer positions at 1, 9, and 17, respectively; (d)–(f) corresponding test accuracy curves for the same cutlayer configurations, "NoComp" denotes the baseline without feature map compression.

TABLE VII
PERPLEXITY DEGRADATION (ΔPPL) UNDER FEATURE COMPRESSION WITHOUT FINE-TUNING RELATIVE TO BASELINE

LLMs	Cutlayers	Feature Compression Methods					
		MS	VQ	QQ	VS	FPQ	RTS
Qwen3-1.7B	Layer 1	**0.61**	9793.42	1478.37	2527.65	93.83	6712.50
	Layer 14	**0.26**	296319000.0	1536123.0	106.52	16753.14	514.18
	Layer 27	**3.72**	37515500.64	79821.40	202.55	229312.64	360.18
Llama3.2-1B	Layer 1	**10.63**	218915.70	34560.14	4397.41	1788204.38	11524.22
	Layer 8	**-5.08**	1925757.87	119130.43	354.18	12992.36	1663.26
	Layer 15	**0.23**	1881815.84	53754.45	325.81	71.10	682.23

TABLE VIII
PERPLEXITY DEGRADATION (ΔPPL) UNDER FEATURE COMPRESSION WITH FINE-TUNING RELATIVE TO BASELINE

LLMs	Cutlayers	Feature Compression Methods					
		MS	VQ	QQ	VS	FPQ	RTS
Qwen3-1.7B	Layer 1	**0.14**	13.72	INF	INF	2.58	INF
	Layer 14	**0.29**	1542.59	151921.1	6.04	INF	18.12
	Layer 27	**0.61**	1434.12	INF	INF	INF	3961.5
Llama3.2-1B	Layer 1	**0.29**	591.39	578.83	1594.8	67.36	1717.51
	Layer 8	**0.12**	1683.81	INF	6.87	665.23	18.93
	Layer 15	**0.43**	1016.62	864.08	12.27	3.43	3473.52

7) NLP Tasks: The experimental results demonstrate MS's superior performance in the NLP model across all test scenarios. Without fine-tuning (Table VII), MS achieves remarkably low perplexity degradation (ΔPPL), with values of 0.61 at Qwen3-1.7B Layer 1 and an actual improvement of −5.08 at Llama3.2-1B Layer 8. In contrast, competing methods exhibit catastrophic performance degradation, with VQ showing ΔPPL up to 296,319,000 and QQ reaching 1,536,123 at Qwen3-1.7B Layer 14. After fine-tuning (Table VIII), MS maintains its performance with ΔPPL consistently below 1.0 across all layers and models. Notably, it achieves 0.14 at Qwen3-1.7B Layer 1 and 0.12 at Llama3.2-1B Layer 8. While FPQ emerges as the second-best method (PPL increased by 2.58 at Qwen3-1.7B Layer 1), most other methods either fail (indicated by ∞ values)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
ACCURACY (%) COMPARISON ACROSS DIFFERENT METHODS UNDER VARYING α (DEGREE OF HETEROGENEITY) AND CUTLAYERS FOR VGG16 TRAINED ON CIFAR10 AND RESNET18 TRAINED ON CIFAR100. "NOCOMP" DENOTES THE BASELINE WITHOUT FEATURE MAP COMPRESSION.

Models	α (heterogeneity)	Cutlayers	Accuracy under Different Methods (%)						
			NoComp	MS	VQ	QQ	VS	FPQ	RTS
VGG16	$\alpha = 1.0$	Layer 1	87.54	**87.25**	86.23	86.33	86.40	86.95	85.41
		Layer 7		**87.47**	87.00	86.99	86.00	87.24	86.73
		Layer 13		87.14	87.01	**87.49**	86.48	87.31	86.77
	$\alpha = 0.5$	Layer 1	82.13	**82.46**	81.01	79.57	80.56	82.07	79.77
		Layer 7		**82.77**	82.13	81.99	81.40	82.24	80.52
		Layer 13		**82.86**	82.38	82.38	81.28	82.13	80.47
	$\alpha = 0.1$	Layer 1	67.77	**67.54**	65.30	65.22	66.07	66.59	64.13
		Layer 7		67.91	**68.74**	67.06	66.92	67.19	66.07
		Layer 13		**68.01**	66.60	67.13	65.50	66.51	66.09
ResNet18	$\alpha = 1.0$	Layer 1	50.56	49.21	47.60	44.64	45.57	**49.99**	42.36
		Layer 9		**49.79**	48.34	49.62	46.46	49.24	47.81
		Layer 17		50.09	50.14	49.48	50.62	49.26	**51.21**
	$\alpha = 0.5$	Layer 1	41.94	42.51	41.15	41.12	37.76	**43.87**	36.64
		Layer 9		**43.45**	43.38	42.84	38.56	43.38	41.47
		Layer 17		42.76	44.07	44.61	44.55	43.38	**44.84**
	$\alpha = 0.1$	Layer 1	41.78	41.03	38.16	39.99	38.54	**41.20**	37.84
		Layer 9		**41.85**	40.77	40.81	39.92	41.14	39.72
		Layer 17		41.31	41.61	41.75	**42.12**	41.23	42.10

or show substantially higher degradation (up to 1,717.51 ΔPPL for RTS at Llama3.2-1B Layer 1).

These results establish MS as the most robust compression method, maintaining stable performance across different model architectures and cutlayers. The performance gap highlights MS's capability to maintain language model quality under compression.

8) Scalability and Heterogeneity: To evaluate the robustness of different compression methods under heterogeneous data distributions, we conduct experiments with varying degrees of heterogeneity controlled by the Dirichlet parameter α. Smaller α corresponds to higher heterogeneity. Following common practice, we simulate a distributed setting with 100 clients to better reflect large-scale and highly heterogeneous scenarios. The results are summarized in Table IX, and the training processes are visualized in Appendix B, available online.

Across different models, cutlayers, and heterogeneity levels, MS consistently achieves the best or near-best performance in most configurations. In particular, under moderate heterogeneity (e.g., $\alpha = 0.5$), MS outperforms all baselines across almost all cutlayers on VGG16. Even under strong heterogeneity ($\alpha = 0.1$), MS maintains superior or competitive accuracy, demonstrating its robustness to non-IID data distributions.

Compared with quantization-based methods (VQ, QQ, FPQ), MS generally achieves higher accuracy due to its lower reconstruction error. Sparsification-based methods (VS, RTS) exhibit more significant performance degradation, especially under high heterogeneity, indicating that directly discarding non-Top-k values is more sensitive to distribution shifts. Although certain methods (e.g., FPQ or QQ) may achieve slightly better results in a few isolated cases, their performance is less consistent across different settings.

As α decreases (i.e., heterogeneity increases), all methods experience performance degradation due to the increased discrepancy between local data distributions. However, MS shows a slower degradation trend compared to other methods, suggesting that reducing compression error is particularly important in heterogeneous settings, where local updates are more sensitive to perturbations.

These results demonstrate that MS is robust under varying degrees of data heterogeneity and can effectively maintain model performance in practical non-IID scenarios. Overall, MS achieves the best performance in the majority of cases, while remaining consistently competitive in all other settings.

9) Summary: The experimental results prove the low compression error, high computational and communication efficiency of MS. Furthermore, the experimental results across both CV and NLP tasks demonstrate the superiority of MS. In CV tasks, MS maintains near-baseline accuracy while achieving higher compression rates than quantization-based methods. These results validate Theorem 4 regarding error propagation in SL, showing that earlier cutlayers are more sensitive to compression errors. For language models, MS shows significantly better perplexity preservation with ΔPPL consistently below 1.0, compared to other methods that sometimes fail. The NLP results partially support our theoretical framework about cutlayer sensitivity. Without fine-tuning, methods exhibit greater error sensitivity at earlier layers: RTS on Qwen3, VS on Llama3.2, FPQ on Llama3.2, and RTS on Llama3.2. These results prove that MS is robust and efficient across different cutlayers and network architectures while validating our Theorem 4 about cutlayer in SL.

VI. CONCLUSION

In conclusion, this paper addresses the critical challenge of communication bottlenecks in SL by proposing the MS compressor. Our method effectively balances the trade-off between communication efficiency and model performance by enhancing classic Top-k sparsification with a lightweight mask encoding,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

thereby significantly reducing compression error under wireless environment. Theoretically, we provide a new analytical perspective by establishing via the mean value theorem, the relationship between compression error and model convergence, as well as between cutlayer selection and output error. These insights not only validate the design principle of our MS compressor but also offer practical guidance for configuring distributed training systems. Extensive experimental results on diverse benchmarks confirm that our approach achieves superior performance, reducing communication overhead while maintaining model accuracy.

Despite these advantages, the study has several limitations. The work does not provide privacy guarantees, and intermediate feature maps remain reconstructible under adversarial attacks. The performance of MS is sensitive to hyperparameters, such as the number of top-k values and the mask width, requiring careful tuning. Cutlayer selection significantly affects error propagation, particularly for shallow layers, which may impact model performance in some scenarios. Finally, U_c and U_s represent theoretical upper bounds, which cannot be computed in practice and are mentioned here only for theoretical context. These limitations highlight directions for future research to enhance privacy, robustness, and cutlayer selection in feature map compression for SL.

REFERENCES

[1] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Conf. Neural Inf. Process. Syst.*, 2022, pp. 27730–27744.

[2] H. Wang et al., "Scientific discovery in the age of artificial intelligence," *Nature*, vol. 620, no. 7972, pp. 47–60, 2023.

[3] P. Zhang, L. Xu, L. Mei, and C. Xu, "Sketch-based adaptive communication optimization in federated learning," *IEEE Trans. Comput.*, vol. 74, no. 1, pp. 170–184, Jan. 2025.

[4] N. Jia, Z. Qu, B. Ye, Y. Wang, S. Hu, and S. Guo, "A comprehensive survey on communication-efficient federated learning in mobile edge environments," in *IEEE Commun. Surveys Tuts.*, vol. 27, no. 6, pp. 3710–3741, Dec. 2025.

[5] Z. Qu et al., "Partial synchronization to accelerate federated learning over relay-assisted edge networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4502–4516, Dec. 2022.

[6] D.-J. Han et al., "Federated split learning with joint personalization-generalization for inference-stage optimization in wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7048–7065, Jun. 2024.

[7] W. Zhou, Z. Qu, Y. Zhao, B. Tang, and B. Ye, "An efficient split learning framework for recurrent neural network in mobile edge environment," in *Proc. Conf. Res. Adaptive Convergent Syst.*, 2022, pp. 131–138.

[8] Z. Lin et al., "Efficient parallel split learning over resource-constrained wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9224–9239, Oct. 2024.

[9] Q. Zhou et al., "Hierarchical channel-spatial encoding for communication-efficient collaborative learning," in *Proc. Conf. Neural Inf. Process. Syst.*, 2022, pp. 5788–5801.

[10] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[11] X. Li and P. Li, "Analysis of error feedback in federated non-convex optimization with biased compression: Fast convergence and partial participation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 19638–19688.

[12] Z. Qu, N. Jia, B. Ye, S. Hu, and S. Guo, "FedQClip: Accelerating federated learning via quantized clipped SGD," *IEEE Trans. Comput.*, vol. 74, no. 2, pp. 717–730, Feb. 2025.

[13] J. Wang et al., "CocktailSGD: Fine-tuning foundation models over 500 Mbps networks," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 36058–36076.

[14] Y. Zhou, M. Shi, Y. Li, Y. Sun, Q. Ye, and J. Lv, "Communication-efficient federated learning with single-step synthetic features compressor for faster convergence," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 5031–5040.

[15] W. Qiu, Y. Zhou, J. Wang, Q. Z. Sheng, and L. Cui, "FLM-TopK: Expediting federated large language model tuning by sparsifying intervalized gradients," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2025, pp. 1–10.

[16] A. Beznosikov, S. Horváth, P. Richtárik, and M. Safaryan, "On biased compression for distributed learning," *J. Mach. Learn. Res.*, vol. 24, no. 276, pp. 1–50, 2023.

[17] Z. Cheng et al., "CHEESE: Distributed clustering-based hybrid federated split learning over edge networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3174–3191, Dec. 2023.

[18] W. Zhou, Z. Qu, S.-H. Lyu, M. Cai, and B. Ye, "Mask-encoded sparsification: Overcoming biased gradients for communication-efficient split learning," in *Proc. Eur. Conf. Artif. Intell.*, 2024, pp. 2806–2813.

[19] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8485–8493.

[20] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018, *arXiv:1812.00564*.

[21] F. Zheng, C. Chen, L. Lyu, and B. Yao, "Reducing communication for split learning by randomized top-K sparsification," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 4665–4673.

[22] R. Dorfman, S. Vargaftik, Y. Ben-Itzhak, and K. Y. Levy, "DoCoFL: Downlink compression for cross-device federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 8356–8388.

[23] H. Liu, F. He, and G. Cao, "Communication-efficient federated learning for heterogeneous edge devices based on adaptive gradient quantization," in *Proc. IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.

[24] C. Yang et al., "Communication-efficient satellite-ground federated learning through progressive weight quantization," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8999–9011, Sep. 2024.

[25] A. Abernathy and M. E. Celebi, "The incremental online k-means clustering algorithm and its application to color quantization," *Expert Syst. Appl.*, vol. 207, 2022, Art. no. 117927.

[26] G. Wang, Q. Qi, R. Han, L. Bai, and J. Choi, "P2CEFL: Privacy-preserving and communication efficient federated learning with sparse gradient and dithering quantization," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14722–14736, Dec. 2024.

[27] D. Alistarh, T. Hoefler, M. Johansson, N. Konstantinov, S. Khirirat, and C. Renggli, "The convergence of sparsified gradient methods," in *Proc. Conf. Neural Inf. Process. Syst.*, 2018, pp. 5977–5987.

[28] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Proc. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1707–1718.

[29] J. Wangni, J. Wang, J. Liu, and T. Zhang, "Gradient sparsification for communication-efficient distributed optimization," in *Proc. Conf. Neural Inf. Process. Syst.*, 2018, pp. 1306–1316.

[30] Y. Matsubara, R. Yang, M. Levorato, and S. Mandt, "Supervised compression for resource-constrained edge computing systems," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2022, pp. 2685–2695.

[31] A. Furutanpey, P. Raith, and S. Dustdar, "FrankenSplit: Efficient neural feature compression with shallow variational bottleneck injection for mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 10770–10786, Dec. 2024.

[32] A. Furutanpey, Q. Zhang, P. Raith, T. Pfandzelter, S. Wang, and S. Dustdar, "FOOL: Addressing the downlink bottleneck in satellite computing with neural feature compression," *IEEE Trans. Mobile Comput.*, vol. 24, no. 8, pp. 6747–6764, Aug. 2025.

[33] Z. Fang et al., "Task-oriented communications for visual navigation with edge-aerial collaboration in low altitude economy," in *Proc. IEEE Glob. Commun. Conf.*, 2025, pp. 1059–1064.

[34] C. Yang et al., "Toward efficient satellite computing through adaptive compression," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 4411–4422, Nov./Dec. 2024.

[35] J. Yang and Y. Liu, "GAS: Generative activation-aided asynchronous split federated learning," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 20, pp. 21956–21964.

[36] Z. Xia et al., "MultiSFL: Towards accurate split federated learning via multi-model aggregation and knowledge replay," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 1, pp. 914–922.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.

[37] D. Basu, D. Data, C. Karakus, and S. N. Diggavi, "Qsparse-local-SGD: Distributed SGD with quantization, sparsification, and local computations," *IEEE J. Sel. Areas Inf. Theory*, vol. 1, no. 1, pp. 217–226, May 2020.
[38] D. Basu, D. Data, C. Karakus, and S. Diggavi, "Qsparse-local-SGD: Distributed SGD with quantization, sparsification and local computations," in *Proc. Conf. Neural Inf. Process. Syst.*, 2019, pp. 1–12.
[39] J. Guo et al., "Compressing large language models by joint sparsification and quantization," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 1–13.
[40] W. Xie et al., "JointSQ: Joint sparsification-quantization for distributed learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 5778–5787.
[41] K. Gruntkowska, A. Tyurin, and P. Richtárik, "EF21-P and friends: Improved theoretical communication complexity for distributed optimization with bidirectional compression," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 11761–11807.
[42] I. Markov, A. Vladu, Q. Guo, and D. Alistarh, "Quantized distributed training of large models with convergence guarantees," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 24020–24044.
[43] Z. Lin et al., "Hierarchical split federated learning: Convergence analysis and system optimization," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 9352–9367, Oct. 2025.
[44] S. Wang, Y. Xu, C. You, M. Shao, and T. Q. S. Quek, "Communication-efficient federated learning by quantized variance reduction for heterogeneous wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 8, pp. 7632–7647, Aug. 2025.
[45] E. Yu, S. Liu, Q. Li, H. Chen, H. V. Poor, and S. Shamai, "Graph-based joint client clustering and resource allocation for wireless distributed learning: A new hierarchical federated learning framework with non-IID data," *IEEE Trans. Mobile Comput.*, vol. 24, no. 5, pp. 3579–3596, May 2025.
[46] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLORA: Efficient finetuning of quantized LLMs," in *Proc. Conf. Neural Inf. Process. Syst.*, 2023, pp. 10088–10115.
[47] S.-y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, "LLM-FP4: 4-bit floating-point quantized transformers," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 592–605.
[48] H. Xu et al., "SLAMB: Accelerated large batch training with sparse communication," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 38801–38825.
[49] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–14.
[50] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[51] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–15
[52] A. Yang et al., "Qwen3 technical report," 2025, *arXiv:2505.09388*.
[53] A. Grattafiori et al., "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.



Wenxuan Zhou received the BS degree in computer science and technology from the Nanjing University of Finance & Economics in 2021. He is currently working toward the PhD degree in computer science and technology with Hohai University. His research interests include edge intelligence and distributed machine learning.



Zhihao Qu (Member, IEEE) received the BSc degree and PhD degree in computer science and technology from Nanjing University in 2009 and 2018, respectively. He is currently an associate professor with the College of Computer Science and Software Engineering, Hohai University, Nanjing, China. He has authored more than 70 top-tier journal and conference papers in related fields, such as INFOCOM, ICML, NeurIPS, ATC, VLDB, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Computers*, and *IEEE Transactions on Services Computing*. His research interests include edge computing and federated learning.



Baoliu Ye (Member, IEEE) received the PhD degree in computer science and technology from Nanjing University, China in 2004. From 2005 to 2006, he was a visiting researcher with the University of Aizu, Japan, and dean of the School of Computer and Information, Hohai University from 2018 to 2023. He is currently a full professor with the School of Computer Science, Nanjing University. His research interests include distributed systems, cloud computing, edge intelligence with more than 100 papers published in major conferences and journals. He is the regent of CCF, and vice-director of CCF Technical Committee of Distributed Computing and Systems.



Shen-Huan Lyu received the BSc degree in statistics from the University of Science and Technology of China, Hefei, China, in 2017, and the PhD degree in computer science from Nanjing University, Nanjing, China, in 2022. He is currently an assistant researcher with the School of Computer Science and Software Engineering, Hohai University. His research interests include machine learning and data mining.



Miao Cai (Member, IEEE) received the PhD degree in computer science and technology from Nanjing University in 2020. From 2018 to 2019, he was a visiting PhD student with the University of Illinois at Urbana-Champaign. He is currently an associate professor with the Nanjing University of Aeronautics and Astronautics. His research interests include storage system and operating system.



Mingtao Ji (Member, IEEE) received the PhD degree from Nanjing University under the supervision of Prof. Baoliu Ye. He is currently a postdoctoral fellow with Hohai University. He has already author or coauthor more than 17 papers, including in journals, such as *IEEE Transactions on Mobile Computing*, *IEEE/ACM Transactions on Networking*, COMNET, JCST, and conferences, such as IEEE INFOCOM, IEEE SECON, IEEE ICPADS, IEEE ISPA, IEEE ICC, IEEE/ACM IWQOS poster, and BigCom. His research interests include edge inference, CPN, P4 switch, SDN, federated learning, Big Data analytics, and distributed machine learning. He won the Best Paper Award in BigCom 2023.



Song Guo (Fellow, IEEE) is currently a full professor with the Department of Computer Science and Engineering, The Hong Kong University of Science and Technology. He is also the Changjiang chair professor awarded by the Ministry of Education of China. His research interests include edge AI, mobile computing, and distributed systems. He has been recognized as a highly cited researcher (Web of Science) and was the recipient of more than 14 best paper awards from IEEE/ACM conferences, journals, and technical committees. He is the editor-in-chief of *IEEE Transactions on Cloud Computing* and founding editor-in-chief of *IEEE Open Journal of the Computer Society*, and associate editor for *IEEE Transactions on Computers* and *ACM Computing Surveys*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 02,2026 at 04:56:52 UTC from IEEE Xplore. Restrictions apply.